

Подготовка к зачёту по C++: Модуль 3

Подробные ответы на вопросы

Студент МФТИ

20 мая 2026 г.

Содержание

1	Идеальная передача и вариативные шаблоны	3
1.1	lvalue-выражения и rvalue-выражения	3
1.2	lvalue-ссылки и rvalue-ссылки	3
1.3	Универсальные ссылки (Forwarding References)	4
1.4	Типы передачи объекта в функцию	5
1.5	Идеальная передача (Perfect Forwarding)	6
1.6	Вариативные шаблоны	6
2	Основы многопоточного программирования. Потоки.	7
2.1	Параллелизм и конкурентность	7
2.2	Потоки. Класс <code>std::thread</code>	8
2.3	Возврат данных из функции потока	9
2.4	Другое	9
3	Мьютексы	10
3.1	Состояние гонки	10
3.2	Стандартный мьютекс <code>std::mutex</code>	10
3.3	Классы <code>lock_guard</code> и <code>unique_lock</code>	10
3.4	Взаимоблокировка (Deadlock)	11
3.5	Защита при инициализации	11
4	Механизмы синхронизации	11
4.1	Условные переменные	11
4.2	Запуск асинхронной задачи	12
4.3	Класс <code>packaged_task</code>	12
4.4	Класс <code>promise</code>	12
5	Потокобезопасные стек и очередь с блокировками	12
5.1	Потокобезопасные структуры данных	12
5.2	Недостатки <code>std::stack</code>	13
5.3	Потокобезопасный стек с блокировками	13
5.4	Потокобезопасная очередь с блокировками	13
5.5	Очередь на основе односвязного списка	14

6	Модели памяти	14
6.1	Барьеры памяти	14
6.2	Модели памяти в C++	14
6.3	Атомарные типы	15
7	Потокобезопасные структуры без блокировок (Lock-Free)	15
7.1	Определения	15
7.2	Lock-Free Stack (Treiber Stack)	15
7.3	Управление памятью	16
8	Пул потоков (Thread Pool)	16

1 Идеальная передача и вариативные шаблоны

1.1 lvalue-выражения и rvalue-выражения

Что такое выражение?

В C++ выражение (expression) — это комбинация операторов и операндов, которая вычисляет значение. Каждое выражение имеет два важных свойства:

1. **Тип (Type):** статический тип данных (например, `int`, `std::string`).
2. **Категория значения (Value Category):** определяет свойства выражения, такие как возможность взятия адреса, возможность перемещения и т.д.

lvalue (left value)

lvalue-выражение — это выражение, которое идентифицирует объект или функцию, имеющую имя или адрес в памяти. К lvalue можно применить оператор взятия адреса (`&`).

- *Примеры:* имена переменных, результаты разыменования указателя (`*ptr`), вызов функции, возвращающей lvalue-ссылку.
- `int x = 10;` здесь `x` — lvalue.
- `std::cout << "hello";` возвращает ссылку на `cout`, это lvalue.

rvalue (right value)

rvalue-выражение — это временное значение, которое не имеет постоянного адреса в памяти (или адрес нельзя взять стандартным способом). Обычно это результаты вычислений, литералы или временные объекты.

- *Примеры:* литералы (`42`, `"hello"`), результат арифметических операций (`a + b`), вызов функции, возвращающей значение по копии.
- `int y = x + 5;` выражение `x + 5` — rvalue.
- `std::string("temp");` — временный объект, rvalue.

1.2 lvalue-ссылки и rvalue-ссылки

Разница между ссылками

- **lvalue-ссылка (T&):** может связываться только с lvalue. Не может связываться с rvalue (за исключением константной ссылки `const T&`, которая продлевает время жизни временного объекта).
- **rvalue-ссылка (T&&):** может связываться только с rvalue. Используется для реализации семантики перемещения (move semantics).

Зачем нужно разделение?

Разделение позволяет оптимизировать код. Если мы знаем, что объект временный (rvalue), мы можем "украсть" его ресурсы (память, файловые дескрипторы) вместо дорогого копирования. Это основа **семантики перемещения**.

Перегрузка по категории выражения

Мы можем перегружать функции, чтобы они по-разному обрабатывали lvalue и rvalue:

```
1 void process(const std::string& s) {
2     std::cout << "Copy: " << s << std::endl;
3 }
4
5 void process(std::string&& s) {
6     std::cout << "Move: " << s << std::endl;
7 }
```

Функция печати категории выражения

Для определения категории можно использовать traits из заголовочного файла `<type_traits>`:

```
1 #include <iostream>
2 #include <type_traits>
3 #include <utility>
4
5 template<typename T>
6 void print_category(T&& t) {
7     if (std::is_lvalue_reference<T>::value) {
8         std::cout << "lvalue expression" << std::endl;
9     } else if (std::is_rvalue_reference<T>::value) {
10        std::cout << "xvalue expression (rvalue ref)" << std::endl;
11    } else {
12        std::cout << "prvalue expression (pure rvalue)" << std::endl;
13    }
14 }
```

Категория имени rvalue-ссылки

Это важный нюанс: **имя** rvalue-ссылки внутри функции является **lvalue**.

```
1 void foo(std::string&& str) {
2     // str - это имя переменной, поэтому оно lvalue,
3     // хотя её тип - rvalue reference.
4     // Чтобы передать её дальше как rvalue, нужен std::move(str).
5 }
```

Что делает `std::move`?

`std::move` не перемещает данные. Она просто выполняет приведение типа (cast) своего аргумента к rvalue-ссылке. Это сигнал компилятору: "Я больше не нуждаюсь в этом объекте, используй его ресурсы". Реализация (упрощенно):

```
1 template<typename T>
2 typename std::remove_reference<T>::type&& move(T&& arg) {
3     return static_cast<typename std::remove_reference<T>::type&&>(arg);
4 }
```

1.3 Универсальные ссылки (Forwarding References)

Правила вывода типов

В шаблонных функциях тип параметра шаблона выводится на основе переданного аргумента. Если аргумент — lvalue типа `T`, то параметр шаблона выводится как `T&`. Если аргумент — rvalue типа `T`, то параметр шаблона выводится как `T`.

Правила свёртки ссылок (Reference Collapsing)

В C++ запрещены ссылки на ссылки, но компилятор использует правила свёртки:

- $T\& \& \rightarrow T\&$
- $T\& \&\& \rightarrow T\&$
- $T\&\& \& \rightarrow T\&$
- $T\&\& \&\& \rightarrow T\&\&$

Что такое универсальная ссылка?

Универсальная ссылка (термин Скотта Мейерса, стандарт называет их *forwarding references*) возникает в двух случаях:

1. В шаблонной функции вида `template<typename T> void f(T&& arg);`
2. При использовании `auto&&` (в C++14 и выше).

Она может принять как lvalue, так и rvalue, сохраняя их категорию.

Какой тип выводится?

- Передача lvalue `int x`: T выводится как `int&`. Согласно правилам свёртки, `int& &&` становится `int&`. Аргумент становится lvalue-ссылкой.
- Передача rvalue `5`: T выводится как `int`. Аргумент имеет тип `int&&` (rvalue-ссылка).

1.4 Типы передачи объекта в функцию

1. По значению (копирование): `void f(T t)`.
 - *Плюсы*: Простота, независимость от оригинала.
 - *Минусы*: Дорогое копирование для больших объектов.
2. По значению (перемещение): `void f(T t)` при вызове `f(std::move(obj))`.
 - *Плюсы*: Дешево, если у T есть move-конструктор.
 - *Минусы*: Оригинальный объект становится валидным, но неуказанным состоянием.
3. По lvalue-ссылке: `void f(T& t)`.
 - *Плюсы*: Нет копирования, можно изменять оригинал.
 - *Минусы*: Нельзя передать временные объекты (rvalue).
4. По rvalue-ссылке: `void f(T&& t)`.
 - *Плюсы*: Позволяет реализовать move-семантику внутри функции.
 - *Минусы*: Нельзя передать lvalue.
5. По константной lvalue-ссылке: `void f(const T& t)`.

- *Плюсы:* Универсально (принимает и lvalue, и rvalue), нет копирования, гарантия неизменности.
- *Минусы:* Нельзя изменять объект внутри.

6. По универсальной ссылке: `template<typename T> void f(T&& t).`

- *Плюсы:* Идеальная передача (perfect forwarding). Сохраняется категория и const-квалификатор аргумента. Максимальная эффективность.
- *Минусы:* Сложность реализации (нужен `std::forward`), код должен быть в заголовочных файлах (шаблоны).

1.5 Идеальная передача (Perfect Forwarding)

Функция `std::forward`

`std::forward` предназначена для передачи аргументов из одной шаблонной функции в другую с сохранением их исходной категории значения (lvalue/rvalue).

Отличие от `std::move`

- `std::move` всегда приводит к rvalue.
- `std::forward` приводит к rvalue **только если** исходный аргумент был rvalue. Если аргумент был lvalue, `std::forward` вернет lvalue.

Реализация `std::forward`

```
1 template<typename T>
2 T&& forward(typename std::remove_reference<T>::type& arg) {
3     return static_cast<T&&>(arg);
4 }
5 // Существует также перегрузка для const T&, но суть в cast'е.
```

Если T вывелся как U& (был lvalue), то возврат U& && схлопывается в U& (lvalue). Если T вывелся как U (был rvalue), то возврат U&& остается rvalue.

1.6 Вариативные шаблоны

Функция с переменным числом аргументов

Используется синтаксис `template<typename... Args>.`

```
1 template<typename... Args>
2 void print_all(Args... args) {
3     // Рекурсивный распаковщик или fold expression
4 }
```

Пакет параметров и раскрытие

Пакет параметров (`Args...`) — это последовательность типов или значений. Раскрытие пакета происходит при добавлении `...` после имени пакета в контексте, где допускается список (инициализатор, список аргументов функции и т.д.).

Где можно раскрывать пакет?

- Список аргументов функции: `func(args...)`
- Инициализация массива/структуры: `int arr[] = {args...};`
- Наследование: `class X : public Base<Args>... {}`
- Fold expressions (C++17).

Fold Expressions (Выражения свёртки)

Способ компактной обработки пакета параметров бинарным оператором.

```
1 template<typename... Args>
2 auto sum(Args... args) {
3     return (args + ...); // Right fold: a + (b + (c + ...))
4 }
5
6 template<typename... Args>
7 void print(Args... args) {
8     (std::cout << ... << args) << std::endl; // Left fold
9 }
```

sizeof...

Оператор `sizeof...(Args)` возвращает количество элементов в пакете параметров.

Применение с идеальной передачей

Часто используется для создания фабричных функций или обёрток (как `std::make_shared` или конструкторы контейнеров):

```
1 template<typename T, typename... Args>
2 std::unique_ptr<T> make_unique(Args&&... args) {
3     return std::unique_ptr<T>(new T(std::forward<Args>(args)...));
4 }
```

2 Основы многопоточного программирования. Потоки.

2.1 Параллелизм и конкурентность

Определения

- **Конкурентность (Concurrency):** Способность системы выполнять несколько задач в течение одного и того же периода времени (переключение контекста). Не обязательно одновременно.
- **Параллелизм (Parallelism):** Одновременное выполнение нескольких задач (требует многоядерного процессора).

Процесс vs Поток

- **Процесс:** Имеет независимую память (address space). Тяжеловесный, изолированный.
- **Поток (Thread):** Легковесная единица исполнения внутри процесса. Потоки одного процесса разделяют память (кучу, глобальные переменные), но имеют свой стек и регистры.

Преимущества и недостатки

- *Процессы:* Надежность (крах одного не валит другой), простота модели памяти. Минус: медленное межпроцессное взаимодействие (IPC), высокое потребление памяти.
- *Потоки:* Быстрый обмен данными через общую память, низкие накладные расходы на создание. Минус: сложность синхронизации, риск гонок данных и deadlock'ов.

2.2 Потоки. Класс `std::thread`

Создание потока

```
1 #include <thread>
2
3 void my_func(int x) { /*...*/ }
4
5 std::thread t(my_func, 42); // Запуск потока
```

Методы `join` и `detach`

- `t.join()`: Текущий поток блокируется и ждет завершения потока `t`. После `join` ресурс потока освобождается.
- `t.detach()`: Поток отделяется и выполняется самостоятельно ("демон"). Объект `std::thread` теряет связь с ним. `Detach` делать опасно, если поток использует локальные переменные создавшей функции.

Важно: Если объект `std::thread` уничтожается (выход из области видимости), а он ни `join`, ни `detach`, вызывается `std::terminate`.

Исключения

Если в новом потоке выбрасывается исключение и не ловится внутри него, вызывается `std::terminate`. Исключение не переходит автоматически в создавший поток. Если в создающем потоке после запуска `t` выбрасывается исключение до `join`, деструктор `t` вызовет `terminate`. Поэтому нужно использовать RAII-обёртки для потоков или блоки `try-catch`.

Передача аргументов

Аргументы копируются во внутреннее хранилище потока. Если нужно передать по ссылке, см. ниже.

2.3 Возврат данных из функции потока

Проблема с обычными ссылками

Конструктор `std::thread` копирует аргументы. Если передать ссылку `T&`, она скопируется как значение, и функция потока будет работать с копией, а не с оригиналом.

Использование `reference_wrapper` и `ref`

`std::ref(x)` возвращает объект `std::reference_wrapper<T>`, который копируем, но внутри хранит ссылку на `x`.

```
1 int result = 0;
2 std::thread t([](int& res){ res = 42; }, std::ref(result));
3 t.join();
4 // result == 42
```

Отличие: `reference_wrapper` ведет себя как ссылка, но является полноценным объектом, который можно копировать и хранить в контейнерах.

2.4 Другое

Идентификация потоков

`std::this_thread::get_id()` возвращает уникальный ID потока. Можно сравнивать ID (`==`, `!=`).

Передача владения

`std::thread` не копируемый, но перемещаемый.

```
1 std::thread t1(func);
2 std::thread t2 = std::move(t1); // t1 теперь пустой, t2 владеет потоком
```

Произвольное количество потоков

Можно хранить потоки в `std::vector<std::thread>`.

```
1 std::vector<std::thread> threads;
2 for (int i = 0; i < 10; ++i) {
3     threads.emplace_back(worker_func, i);
4 }
5 for (auto& t : threads) {
6     if (t.joinable()) t.join();
7 }
```

Использование `mem_fn`

Адаптер, позволяющий вызывать метод класса как функцию. Полезно при передаче методов в потоки.

```
1 struct Foo { void bar() {} };
2 Foo f;
3 std::thread t(std::mem_fn(&Foo::bar), &f);
```

3 Мьютексы

3.1 Состояние гонки

Разделяемые данные

Данные, к которым обращаются несколько потоков одновременно, причем хотя бы один поток пишет.

Race Condition vs Data Race

- **Data Race (Гонка данных):** Два потока обращаются к одной ячейке памяти одновременно, хотя бы одна запись, и нет синхронизации. Это **Undefined Behavior (UB)** в C++.
- **Race Condition (Состояние гонки):** Более широкое понятие. Логическая ошибка, когда результат выполнения зависит от порядка исполнения потоков. Не всегда UB, но часто баг.

Причины и последствия

Возникает при отсутствии атомарности операций чтения-изменения-записи. Приводит к повреждению данных, крашам, непредсказуемому поведению.

3.2 Стандартный мьютекс `std::mutex`

`std::mutex` обеспечивает эксклюзивный доступ.

- `lock()`: Блокирует мьютекс. Если занят, поток засыпает.
- `unlock()`: Освобождает мьютекс.
- `try_lock()`: Пытается захватить, не блокируя. Возвращает `bool`.

3.3 Классы `lock_guard` и `unique_lock`

Недостатки raw mutex

Если забыть вызвать `unlock` (например, из-за исключения или раннего `return`), мьютекс останется заблокированным навсегда (deadlock для других потоков).

Использование `lock_guard`

RAII-обёртка. Конструктор вызывает `lock()`, деструктор `unlock()`.

```
1 {  
2     std::lock_guard<std::mutex> guard(mtx);  
3     // работа с данными  
4 } // автоматический unlock
```

Плюс: Простота, безопасность исключений. *Минус:* Нельзя явно разблокировать раньше конца scope, нельзя передать владение.

Использование `unique_lock`

Более гибкая RAII-обёртка.

- Поддерживает отложенную блокировку (`std::defer_lock`).
- Поддерживает явный `unlock()` и повторный `lock()`.
- Поддерживает перемещение владения мьютексом.
- Необходим для работы с `std::condition_variable`.

Минус: Чуть больше накладных расходов, чем у `lock_guard`.

3.4 Взаимоблокировка (Deadlock)

Возникает, когда два потока ждут ресурсы, захваченные друг другом. **Решение:** `std::lock` (функция, не метод). Она блокирует несколько мьютексов одновременно, используя алгоритм предотвращения deadlock (например, try-lock loop).

```
1 std::lock(mutex1, mutex2);
2 std::lock_guard<std::mutex> lk1(mutex1, std::adopt_lock);
3 std::lock_guard<std::mutex> lk2(mutex2, std::adopt_lock);
```

Или использовать `std::scoped_lock` (C++17), который делает то же самое проще.

3.5 Защита при инициализации

Double-Checked Locking

Паттерн для ленивой инициализации синглтона. Сначала проверка без локировки, потом с локировкой. В C++11+ лучше использовать `std::call_once`.

Использование `call_once`

Гарантирует, что функция будет вызвана ровно один раз, даже при многопоточном доступе.

```
1 std::once_flag flag;
2 void init() { /* дорогая инициализация */ }
3
4 void thread_func() {
5     std::call_once(flag, init);
6 }
```

4 Механизмы синхронизации

4.1 Условные переменные

`std::condition_variable` позволяет потоку заснуть до наступления определенного условия.

- `wait(unique_lock& lk)`: Атомарно разблокирует мьютекс и усыпляет поток. При пробуждении снова блокирует мьютекс.
- `notify_one()`: Будит один ожидающий поток.
- `notify_all()`: Будит все ожидающие потоки.

Ложные пробуждения (Spurious Wakeups)

Поток может проснуться без вызова `notify`. Поэтому `wait` всегда нужно вызывать в цикле с проверкой предиката:

```
1 cv.wait(lk, []{ return data_ready; }); // Эквивалентно циклу while(!
    data_ready) wait(lk);
```

4.2 Запуск асинхронной задачи

`std::async` запускает функцию асинхронно (в новом потоке или отложено, зависит от политики запуска). Возвращает `std::future<T>`, который хранит результат.

```
1 auto fut = std::async(std::launch::async, [](){ return 42; });
2 int res = fut.get(); // Блокирует поток до получения результата
```

4.3 Класс `packaged_task`

Обёртка вокруг вызываемого объекта, которая связывает его с `std::future`. Позволяет вручную управлять тем, *где* и *когда* будет выполнена задача (например, передать задачу в пул потоков).

```
1 std::packaged_task<int> task([](){ return 42; });
2 std::future<int> fut = task.get_future();
3 std::thread t(std::move(task)); // Запуск в потоке
4 t.detach();
5 int res = fut.get();
```

4.4 Класс `promise`

Низкоуровневый механизм для установки значения в `future`. Полезен, когда результат вычисляется не в виде возврата функции, а приходит из callback'a или другого события.

```
1 std::promise<int> prom;
2 std::future<int> fut = prom.get_future();
3
4 std::thread t([&prom]() {
5     prom.set_value(42); // Установка значения
6 });
7
8 int res = fut.get();
9 t.join();
```

5 Потокобезопасные стек и очередь с блокировками

5.1 Потокобезопасные структуры данных

Структура данных, методы которой можно безопасно вызывать из нескольких потоков одновременно без внешней синхронизации. **С блокировками:** Внутри каждого метода используется мьютекс.

STL контейнеры

Стандартные контейнеры (`std::vector`, `std::map` и т.д.) **НЕ** являются потокобезопасными. Даже чтение одним потоком и запись другим — это Data Race.

5.2 Недостатки `std::stack`

Интерфейс `std::stack` разделяет операции `top()` (чтение вершины) и `pop()` (удаление вершины). В многопоточной среде это создает проблему:

1. Поток 1 вызывает `top()`, получает ссылку.
2. Поток 2 вызывает `pop()`, удаляет элемент.
3. Поток 1 обращается по ссылке — **Use After Free / UB**.

Нужна атомарная операция "получить и удалить".

5.3 Потокобезопасный стек с блокировками

Реализация должна объединять `top` и `pop` в одну операцию `try_pop`, возвращающую `optional` или использующую выходной параметр.

```
1 template<typename T>
2 class ThreadSafeStack {
3 private:
4     std::stack<T> data;
5     mutable std::mutex mtx;
6
7 public:
8     void push(T val) {
9         std::lock_guard<std::mutex> lk(mtx);
10        data.push(std::move(val));
11    }
12
13    bool try_pop(T& value) {
14        std::lock_guard<std::mutex> lk(mtx);
15        if (data.empty()) return false;
16        value = std::move(data.top());
17        data.pop();
18        return true;
19    }
20 };
```

Exception Safety: Если копирование `data.top()` в `value` выбросит исключение, стек останется в консистентном состоянии (элемент не удален). Если использовать перемещение, оно обычно поехсепт.

5.4 Потокобезопасная очередь с блокировками

Методы:

- `push`: добавляет элемент.
- `try_pop`: пытается забрать, возвращает `false` если пусто.
- `wait_and_pop`: ждет, пока очередь не станет непустой.

Для `wait_and_pop` нужна условная переменная.

```
1 template<typename T>
2 class ThreadSafeQueue {
3 private:
4     std::queue<T> data;
5     mutable std::mutex mtx;
```

```

6     std::condition_variable cv;
7
8 public:
9     void push(T val) {
10         {
11             std::lock_guard<std::mutex> lk(mtx);
12             data.push(std::move(val));
13         }
14         cv.notify_one(); // Уведомляем вне локировки (оптимизация)
15     }
16
17     void wait_and_pop(T& value) {
18         std::unique_lock<std::mutex> lk(mtx);
19         cv.wait(lk, [this]{ return !data.empty(); });
20         value = std::move(data.front());
21         data.pop();
22     }
23
24     bool try_pop(T& value) {
25         std::lock_guard<std::mutex> lk(mtx);
26         if(data.empty()) return false;
27         value = std::move(data.front());
28         data.pop();
29         return true;
30     }
31 };

```

5.5 Очередь на основе односвязного списка

Идея: использовать два мьютекса. Один защищает голову (для pop), другой хвост (для push). Это увеличивает параллелизм, так как push и pop могут выполняться одновременно, если очередь не пуста/не пуста до конца. Требуется аккуратная работа с указателями и dummy-узлами, чтобы избежать гонок при переходе от состояния "один элемент" к "пусто" или "два элемента".

6 Модели памяти

6.1 Барьеры памяти

Процессор и компилятор могут переупорядочивать инструкции для оптимизации.

- **LoadLoad:** Загрузка не может быть перемещена перед другой загрузкой.
- **StoreStore:** Запись не может быть перемещена после другой записи.
- **Acquire:** Запрещает перемещение чтений/записей *после* барьера *до* барьера. (Гарантирует видимость изменений, сделанных другими потоками до release).
- **Release:** Запрещает перемещение чтений/записей *до* барьера *после* барьера. (Гарантирует, что все изменения видны другим потокам после acquire).

6.2 Модели памяти в C++

- **memory_order_seq_cst** (Sequentially Consistent): По умолчанию. Самая строгая, гарантирует единый порядок всех атомарных операций для всех потоков. Дорогая.

- `memory_order_acquire / release`: Используются для пар "производитель-потребитель". Дешевле, чем `seq_cst`.
- `memory_order_relaxed`: Только атомарность самой операции, никаких гарантий упорядочивания относительно других переменных. Самая быстрая. Подходит для счетчиков.

`std::atomic_thread_fence` — явный барьер памяти, не привязанный к конкретной переменной.

6.3 Атомарные типы

`std::atomic<T>` гарантирует, что операции над переменной будут неделимыми.

Использование `atomic_flag`

Единственный lock-free атомарный тип, гарантированный стандартом везде. Имеет методы `test_and_set` (atomic exchange with true) и `clear`.

Реализация спинлока

```

1 class SpinLock {
2     std::atomic_flag flag = ATOMIC_FLAG_INIT;
3 public:
4     void lock() {
5         while (flag.test_and_set(std::memory_order_acquire)) {
6             // busy wait
7         }
8     }
9     void unlock() {
10        flag.clear(std::memory_order_release);
11    }
12 };

```

7 Потокобезопасные структуры без блокировок (Lock-Free)

7.1 Определения

- **Blocking**: Если поток приостанавливается ОС (mutex).
- **Lock-free**: Гарантируется, что хотя бы один поток прогрессирует за конечное число шагов. Обычно реализуется через CAS (Compare-And-Swap).
- **Wait-free**: Каждый поток завершает свою операцию за конечное число шагов, независимо от других. Самый сильный guarantee, сложно реализовать.

7.2 Lock-Free Stack (Treiber Stack)

Использует связный список и атомарный указатель на голову. **Push**:

1. Создать новый узел.

2. В цикле: прочитать текущую голову, установить `next` нового узла в голову, попытаться `CAS(head, old_head, new_node)`.

Pop:

1. В цикле: прочитать голову. Если `NULL`, стек пуст.
2. Прочитать `next` головы.
3. `CAS(head, old_head, next)`. Если успешно, вернуть данные старого `head`.

Проблема АВА: Если поток считал голову А, был вытеснен, другой поток удалил А, добавил В, потом снова А (на том же адресе памяти). Первый поток проснется и сделает `CAS` успешно, но структура нарушена. Решается Hazard Pointers или счетчиками версий (DCAS).

7.3 Управление памятью

В lock-free структурах нельзя просто делать `delete` узла, который мог быть прочитан другим потоком.

- **Hazard Pointers:** Каждый поток объявляет указатели, которые он сейчас "использует". Память освобождается только если никто не ссылается на узел через hazard pointer.
- **Reference Counting (RCU подобный):** Подсчет ссылок на узлы.

8 Пул потоков (Thread Pool)

Пул потоков — это набор заранее созданных потоков, которые берут задачи из общей очереди. Избегает накладных расходов на создание/уничтожение потоков для каждой короткой задачи.

Реализация:

1. Вектор потоков (`std::vector<std::thread>`).
2. Очередь задач (`std::queue<std::function<void()>`» или `packaged_task`).
3. Мьютекс и condition variable для синхронизации доступа к очереди.
4. Флаг остановки (`bool stop`).

Submit: Добавляет задачу в очередь, уведомляет CV. **Worker Loop:** Поток ждет уведомления, берет задачу из очереди, выполняет её.

Предотвращение конкуренции: Вся очередь защищена мьютексом. Задачи должны быть независимыми.